

OUTILS DE COLLABORATION WEB : Étude exploratoire

Marc Lefrançois

Le-Point-Technique, 06/2024

abstract: Étude exploratoire des solutions de collaboration Web (visioconférence et partage de fichiers) avec analyse du marché, comparaison architecturale et recommandations pour la société Astra Recherche.

keywords: visioconférence, partage de fichiers, architecture logicielle, NextCloud, Jitsi

1. Objet

Le présent document concerne l'étude exploratoire dans le cadre d'une solution Web de collaboration, réunion et présentation pour la société Astra Recherche. Son objectif est de présenter les différentes options d'architecture et d'étudier les différentes solutions pour les composants de visioconférences et de partage de fichier.

2. Situation actuelle

2.1. Diagramme de composant

Veuillez consulter *Figure 1*.

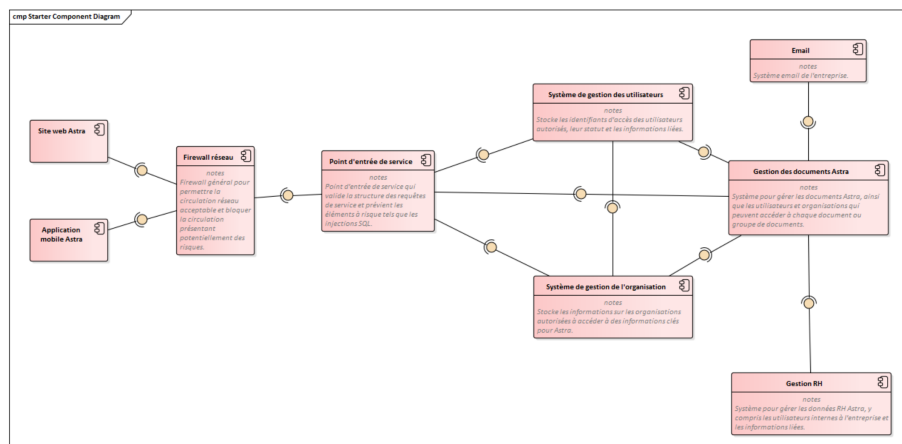


Figure 1: Diagramme de composant

2.2. Description du diagramme de composant

- **Site web Astra** : application web réactive affichant des informations publiques et protégées par login, accessible depuis tout type d'appareil.
- **Application mobile Astra** : application Android/iOS offrant accès mobile aux données Astra et un stockage limité de documents.
- **Firewall réseau** : protège les systèmes Astra et filtre les accès par port 80 pour les services exposés.
- **Point d'entrée de service** : vérifie l'accès des utilisateurs aux services autorisés.
- **Système de gestion des utilisateurs** : gère les permissions, rôles et authentifications.
- **Système de gestion de l'organisation** : gère les organisations autorisées et l'accès aux données associées.
- **Email** : service interne de messagerie et emails transactionnels.
- **Gestion des documents Astra** : protection et accès contrôlé aux documents selon rôles et autorisations.
- **Gestion RH** : gestion des utilisateurs internes (rôle, département, permissions).

2.3. Remarques sur le diagramme de composant

- L'architecture des opérations IT d'Astra comprend 9 composants.
 - Les composants sont fortement couplés.
 - L'architecture semble monolithique.
-

3. Démarche exploratoire

La démarche de l'étude exploratoire a été réalisée selon trois étapes :

1. Étude des options du marché
2. Étude des options architecturales
3. Sélection de la solution retenue

4. Options du marché

4.1. Outil de visioconférence

4.1.1. Cisco WebEx Meetings (Webex Suite) **Description** : solution clé en main (appels audio/vidéo, visioconférence, messagerie instantanée, sondages, événements).

Avantages : support, compatibilité multiplateforme, chiffrement de bout en bout, conformité RGPD, API d'extension, choix de la région de stockage.

Inconvénients : propriétaire, données stockées chez un tiers, coût mensuel (30–50€/licence), pas de support Linux.

Conclusion : 90 % des besoins couverts mais non retenue (coût + absence d'intégration).

4.1.2. Zoom **Description** : messagerie et visioconférence (appels, messagerie, calendrier, événements).

Avantages : support, mise en place rapide, API, chiffrement, conformité RGPD.

Inconvénients : propriétaire, données chez un tiers, coût (17€/licence ou plus), pas de Linux, suspicion sur la collecte des données.

Conclusion : 85 % des besoins couverts mais non retenue (coût + confidentialité).

4.1.3. Microsoft Teams **Description** : communication collaborative intégrée à Microsoft 365.

Avantages : support, interconnectivité dans l'écosystème Microsoft, chiffrement, conformité RGPD.

Inconvénients : propriétaire, dépendance à Microsoft 365, coût (~12€/licence), pas d'enregistrement natif, pas d'API d'extension.

Conclusion : 80 % des besoins couverts mais non retenue (coût + verrou Microsoft).

4.1.4. Solution applicative Google **Description** : Google Meet (visioconférence) + Google Chat (messagerie).

Avantages : solution clé en main, chiffrement, conformité RGPD.

Inconvénients : propriétaire, collecte de données possible, coût (~12€/licence), dépendance au cloud Google.

Conclusion : 95 % des besoins couverts mais non retenue (coût + intégration limitée).

4.1.5. Jitsi **Description** : application libre (OpenSource) pouvant fonctionner en JaaS.

Avantages : marque blanche, coût maîtrisé, données internes, extensible via API et code source, adoption étatique française, pas de limite logicielle de participants.

Inconvénients : nécessite des moyens d'intégration et de maintenance supplémentaires.

Conclusion : répond à 100 % des besoins, d'où le statut **retenue**.

4.1.6. NextCloud Talk **Description** : extension NextCloud pour la collaboration (appels, visioconférence, messagerie).

Avantages : OpenSource, marque blanche, maîtrise des coûts, extensible, données internes, adoption ministérielle, pas de limitation logicielle.

Inconvénients : besoin de moyens pour intégration et maintien.

Conclusion : répond à 98 % des besoins, d'où le statut **retenue**.

4.2. Outil de partage de fichiers

4.2.1. Google Drive **Avantages** : intégration avec de nombreux outils, chiffrement, conformité RGPD, antivirus intégré.

Inconvénients : propriétaire, données chez un tiers, coût mensuel, suspicion sur la collecte de données.

Conclusion : non retenue.

4.2.2. NextCloud Files **Avantages** : OpenSource, marque blanche, extensible, chiffrement, conformité RGPD, adoption ministérielle.

Inconvénients : nécessite des moyens d'intégration et de maintenance.

Conclusion : répond à 98 % des besoins, d'où le statut **retenue**.

4.2.3. SharePoint **Avantages** : intégration Microsoft, extensible, déploiement interne possible.

Inconvénients : propriétaire, dépendance Microsoft, offre trop large, coût mensuel (~28€/licence), pas Linux.

Conclusion : non retenue.

4.2.4. Liferay **Avantages** : OpenSource, extensible en JEE, coûts maîtrisés.

Inconvénients : nécessite des compétences spécifiques et un temps d'implémentation élevé.

Conclusion : non retenue.

4.3. Conclusion

Solutions retenues :

- **Visioconférence** : Jitsi ou NextCloud Talk
 - **Partage de fichiers** : NextCloud Files
-

5. Options architecturales

5.1. Critères architecturaux

- **Évolutivité** : ajout efficace de nouvelles solutions.
- **Simplicité** : bonne granularité, documentation à jour, réutilisabilité.
- **Maintenabilité** : support du MCO, outils d'investigation, évolutivité.
- **Compatibilité** : avec les plateformes matérielles et logicielles.
- **Interconnectivité** : standards d'interfaçage (ETL, Web services).

5.2. Options architecturales

5.2.1. Typologie architecturale (Voir tableau dans Table 1 ci-dessous)

ARCHITECTURE	EXEMPLE D'APPLICATION
Client-serveur	
Pilotée par événements	Micro-blogging, automatisation d'usine
Orientée services	Suivi de colis, validation bancaire
Modulaire	Progiciel avec extensions VBA
En couches	Pile TCP/IP, DAO
Centrée sur les données	CRM

5.2.2. Architecture client-serveur Répartit les tâches entre les fournisseurs de ressources (serveurs) et les consommateurs de ces ressources (clients), qui y accèdent par le biais d'ordinateurs, tablettes ou smartphones. Ce type d'architecture centralise et facilite l'accès aux ressources, tout en séparant les aspects matériel, logiciel et fonctionnel, ce qui en facilite la maintenance.

5.2.3. Architecture pilotée par les événements S'organise autour d'un bus d'événements informationnels, avec en entrée les différents types d'événements survenant (les déclencheurs) et en sortie les canaux correspondant à la nature de l'événement. Elle permet de répondre à des situations non prédictives, en adoptant un comportement spécifique à chaque événement.

5.2.4. Architecture orientée services Centralise des services Web internes (site Web, Extranet, Intranet) ou externes (fournisseurs indépendants tels que

des transporteurs ou des banques), afin de les mettre à disposition des clients en toute transparence et de faciliter leur expérience utilisateur.

5.2.5. Architecture modulaire S'appuie sur un dispositif permettant d'ajouter des fonctionnalités non prévues initialement, en étendant les fonctionnalités de base afin de les spécialiser selon les besoins des utilisateurs, tout en facilitant la maintenance et la réutilisabilité.

5.2.6. Architecture en couches S'articule autour d'une pile organisée par thèmes fonctionnels, afin de traiter indépendamment chaque aspect dans le cadre de processus verticaux, ce qui facilite également la maintenance.

5.2.7. Architecture centrée sur les données Permet l'implémentation de stratégies (par exemple commerciales) en gérant les règles de gestion dans des tableaux plutôt que dans le code, ce qui facilite la maintenabilité : la logique du système est basée sur ses données plutôt que sur des règles fixées intrinsèquement par le code.

5.3. Architecture cible de visioconférence

5.3.1. Architectures non retenues

- **Client-serveur** : il ne s'agit pas de mise à disposition de ressources centralisées, mais de diffusion d'informations (voix, vidéos, texte, fichiers).
- **Orientée services** : il ne s'agit pas de mise à disposition de services internes ou externes, mais de diffusion d'informations.
- **Modulaire** : le système actuel ne dispose d'aucun dispositif visant à étendre ses fonctionnalités ; ce n'est donc pas, à proprement parler, une architecture modulaire.
- **En couches** : le besoin de visioconférence requiert une architecture plutôt horizontale que verticale (diffuser de l'information d'un point vers un ou plusieurs autres), au moins pour l'aspect logiciel — l'architecture technique (protocole TCP/IP) pourrait, elle, s'en accommoder.
- **Centrée sur les données** : le besoin ne comporte pas d'implémentation de stratégies métier visant à orienter les utilisateurs.

5.3.2. Architecture retenue Il s'agit de l'architecture pilotée par événements. De par sa composition (bus, événements, canaux de diffusion) et son objectif de répondre à la survenue d'événements non prédictifs par l'adoption d'un comportement informatif spécifique à chacun d'eux, elle est adaptée à la diffusion d'informations dédiées à des groupes d'utilisateurs identifiés, et remplit donc les fonctions attendues de l'outil de visioconférence.

5.4. Architecture cible de partage de fichiers

5.4.1. Architectures non retenues

- **Pilotée par événements** : il ne s'agit pas d'une situation non prédictive à laquelle répondre par la diffusion d'informations à un groupe d'utilisateurs — c'est l'inverse : l'utilisateur accède, selon son niveau d'autorisation, à un ensemble de ressources (ajout, modification, suppression) sans dépendre du déclenchement d'actions par d'autres.
- **Orientée services** : il s'agit ici de centralisation de ressources documentaires, pas de mise à disposition de services internes ou externes.
- **Modulaire** : de même qu'en 5.3.1, le système actuel n'est pas, à proprement parler, une architecture modulaire.
- **En couches** : le besoin de partage de fichiers requiert une architecture visant à centraliser et mettre à disposition des ressources aux utilisateurs, plutôt qu'une organisation verticale par couches — au moins pour l'aspect logiciel.
- **Centrée sur les données** : le besoin ne comporte pas d'implémentation de stratégies métier visant à orienter les utilisateurs.

5.4.2. Architecture retenue Il s'agit de l'architecture client-serveur. Son objectif — centraliser et faciliter l'accès à des ressources pour des clients (consommateurs) — correspond exactement au besoin de partage de documents énoncé en section 4.

6. Option retenue

6.1. Comparatif interne vs externe

6.1.1. Solution interne Avantages : sur-mesure, maîtrise des coûts, indépendance fournisseur.

Inconvénients : coûts humains/financiers élevés, délais longs, risques sécurité.

6.1.2. Solution externe Avantages : clé en main, support, délais réduits.

Inconvénients : dépendance fournisseur, coût mensuel possible.

6.1.3. Conclusion Choix : **solution externe**.

6.2. Propriétaire vs OpenSource

Conclusion : OpenSource préférable (indépendance, personnalisation, coûts).

6.3. Choix final

Solution retenue : **NextCloud** (NextCloud Talk + NextCloud Files).